LAPORAN INTERNET OF THINGS

PERTEMUAN 9

BASIS DATA UNTUK SISTEM IoT

Nama : Rafiq Alhariri Andriansyah

NIM : 240605110178

Kelas : Internet of Things A

TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM MALANG

GANJIL 2026/2027

DAFTAR ISI

BAB I Pendahuluan

BAB II Rancangan Basis Data

BAB III Implementasi dan Penjelasan Gambar

- 3.1 Kode CREATE TABLE
- 3.2 Kode ingestion yang mengisi tabel
- 3.3 Struktur tabel
- 3.4 Query dan hasilnya
- 3.5 Dashboard

BAB IV Pengujian Validasi, Timestamp, dan Kualitas Data

BAB V Kesimpulan dan Saran

Lampiran: Daftar Berkas

BAB I
PENDAHULUAN

1.1 Latar Belakang

Proyek UTS membangun sistem pemantauan suhu dan kelembapan. ESP32 dengan sensor DHT22 (simulasi Wokwi) mengirim data ke laptop melalui dua protokol yang dibandingkan, yaitu MQTT (broker Mosquitto) dan HTTP (server Flask). Program penerima `server.py` mencatat setiap pembacaan ke SQLite (`data/sensor.db`), menyalinnya ke Supabase, dan menampilkannya pada dashboard real-time.

Pada versi UTS, basis data hanya berupa log pembacaan dan laporan RTT. Belum ada data induk perangkat, relasi antar tabel, maupun aturan validasi di tingkat basis data. Tugas ini melanjutkan proyek tersebut sesuai materi Pertemuan 9: merancang basis data relasional (perangkat, telemetri, dan alarm), menyimpan data melalui alur ingestion yang tervalidasi, menjalankan query analisis, dan menampilkan hasilnya pada dashboard.

1.2 Tujuan

1. Membuat tabel basis data IoT (CREATE TABLE) lengkap dengan primary key, foreign key, constraint, dan indeks.
2. Menyimpan data sensor melalui alur ingestion yang memvalidasi payload sebelum INSERT.
3. Menjalankan query untuk mengambil, meringkas, menggabungkan (JOIN), dan memeriksa kualitas data.
4. Menampilkan isi dan hasil query basis data pada dashboard.

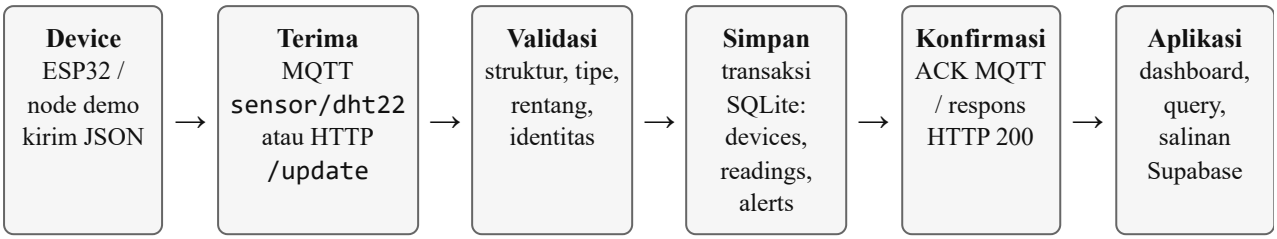
1.3 Lingkungan dan Sumber Data

Komponen	Keterangan
Basis data	SQLite 3.50.4, file `data/sensor.db`, mode jurnal WAL, foreign key aktif
Ingestion	Python 3.13: Flask (HTTP `POST /update`, port 5000) dan paho-mqtt (topik `sensor/dht22`)
Broker MQTT	Mosquitto 2.1.2 pada 127.0.0.1:1883
Dashboard	HTML, CSS, dan JavaScript: `http://127.0.0.1:5000/` dan halaman baru `/database`
Data	2.263 record dari log pengujian UTS (28 September–5 Oktober 2026), ditambah 105 record baru pada 2 November 2026. Total 2.368 pembacaan.

> Data dikirim oleh node demo Python (`demo-node-01`, `source_mode = demo`) yang meniru ESP32. Node ini mengirim bergantian melalui MQTT dan HTTP dengan format payload dan jalur penyimpanan yang sama. Wokwi Private IoT Gateway tidak tersedia, sehingga data bukan hasil ukur DHT22 fisik. Supabase tidak dijalankan pada pengujian ini; data baru tetap aman di SQLite sebagai antrean sinkronisasi.

BAB II
RANCANGAN BASIS DATA

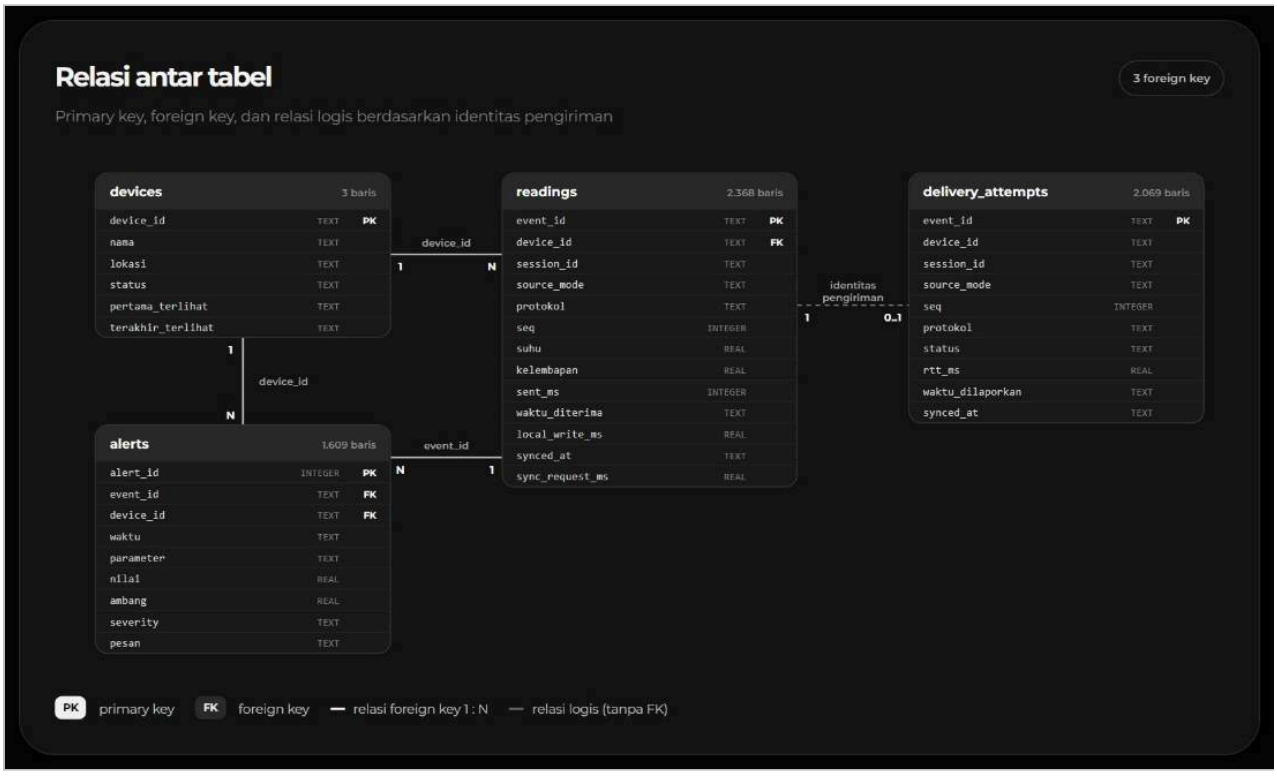
2.1 Alur Ingestion



Gambar 1. Alur ingestion data sensor pada proyek

Alur mengikuti tahapan pada materi (terima → validasi → normalisasi → simpan → konfirmasi). Server menambahkan `event_id` (UUID), `protokol`, dan `waktu_diterima` (UTC). Seluruh penulisan satu pembacaan dilakukan dalam satu transaksi. ACK baru dikirim setelah commit, sehingga data yang sudah dikonfirmasi pasti tersimpan. Pengiriman ulang yang identik dijawab dengan ACK lama tanpa membuat record baru (idempoten).

2.2 Relasi Antar Tabel



Gambar 2. Relasi antar tabel pada halaman Database Explorer

Gambar 2 adalah diagram relasi yang digambar otomatis dari metadata basis data (PK dan FK dibaca dengan `PRAGMA foreign_key_list`). Rancangannya mengikuti skema relasional *Smart Greenhouse* pada materi (DEVICES, SENSOR_READINGS, ALERTS), ditambah satu tabel dari proyek UTS:

- **devices** (3 baris) adalah tabel induk perangkat dengan PK `device_id`. Satu perangkat memiliki banyak pembacaan dan banyak alarm (relasi 1 : N, garis bertanda 1 dan N).
- **readings** (2.368 baris) menyimpan telemetri. PK `event_id`, FK `device_id` → devices.
- **alerts** (1.609 baris) menyimpan alarm. FK `event_id` → readings (pembacaan pemicu) dan FK `device_id` → devices.
- **delivery_attempts** (2.069 baris) menyimpan laporan RTT untuk perbandingan MQTT dan HTTP. Relasinya dengan readings bersifat logis 1 : 0..1 (garis putus-putus) melalui identitas `(device_id, session_id, protokol, seq)`. FK tidak dipakai karena laporan RTT bisa tiba lebih dulu daripada pembacaannya, atau pembacaannya gagal terkirim.

Tabel readings memakai pola time-series di dalam basis data relasional. `waktu_diterima` berperan sebagai timestamp; `device_id`, `session_id`, `source_mode`, dan `protokol` berperan sebagai tag untuk filter; `suhu` dan `kelembapan` berperan sebagai field nilai. Indeks `(device_id, waktu_diterima)` mempercepat query berbasis waktu, sedangkan relasi dan constraint menjaga integritas data perangkat dan alarm.

2.3 Timestamp

- **Ingestion time** – `waktu_diterima` dibuat server saat pesan diterima, sebelum validasi dan penyimpanan. Formatnya ISO 8601 UTC dengan milidetik, misalnya `2026-11-02T04:13:42.297+00:00`. Dashboard dan query mengonversinya ke WIB (UTC+7) hanya saat ditampilkan.
- **Waktu perangkat** – `sent_ms` adalah `millis()` pengirim (lama menyala), bukan waktu absolut. Nilai ini tidak boleh dikurangkan dari waktu UTC server.
- **Event time** absolut belum tersedia karena ESP32 tidak memakai RTC/NTP. Urutan kejadian dalam satu sesi dijaga oleh nomor urut `seq`.
- Constraint `CHECK (waktu_diterima LIKE '____-__-__T__:__:__%+00:00')` menolak timestamp yang tidak berformat UTC. Karena formatnya seragam, urutan teks sama dengan urutan waktu sehingga indeks dapat dipakai untuk filter rentang waktu.

BAB III
IMPLEMENTASI DAN PENJELASAN GAMBAR

3.1 Kode CREATE TABLE

Seluruh tabel didefinisikan dalam satu berkas `database/schema.sql`. Fungsi `init_db()` di `server.py` menjalankan berkas ini setiap kali server dimulai. Semua pernyataan memakai `IF NOT EXISTS` sehingga aman dijalankan berulang kali.

schema.sql — database/schema.sql · baris 1–51

```sql
-- ========================================================================
-- Skema basis data IoT: penerima ESP32 + DHT22 proyek UTS (SQLite 3)
-- File database: data/sensor.db, dijalankan otomatis oleh init_db() di
-- server.py. Waktu disimpan sebagai teks ISO 8601 UTC, misalnya
-- 2026-11-02T03:15:20.123+00:00. Dashboard mengubahnya ke WIB saat tampil.
-- ========================================================================

-- 1) Metadata perangkat. Satu perangkat memiliki banyak pembacaan dan alarm.
create table if not exists devices (
    device_id         text primary key not null      -- ID unik perangkat
                      check (length(device_id) between 1 and 32),
    nama              text not null,                 -- nama tampilan
    lokasi            text,                          -- lokasi pemasangan
    status            text not null default 'aktif'  -- nonaktif: data ditolak
                      check (status in ('aktif', 'nonaktif')),
    pertama_terlihat  text,                          -- pembacaan pertama (UTC)
    terakhir_terlihat text                           -- pembacaan terakhir (UTC)
);

-- 2) Telemetri sensor: satu baris untuk setiap pembacaan yang diterima.
create table if not exists readings (
    event_id        text primary key not null,       -- UUID dari server
    device_id       text not null                    -- pengirim (FK)
                    references devices(device_id),
    session_id      text,                            -- sesi boot pengirim
    source_mode     text not null default 'unknown'  -- asal data
                    check (source_mode in ('dht22', 'random', 'demo', 'unknown')),
    protokol        text not null                    -- jalur pengiriman
                    check (protokol in ('MQTT', 'HTTP')),
    seq             integer not null                 -- nomor urut per sesi
                    check (seq between 0 and 4294967295),
    suhu            real not null                    -- derajat Celsius
                    check (suhu between -40 and 125),
    kelembapan      real not null                    -- persen RH
                    check (kelembapan between 0 and 100),
    sent_ms         integer not null                 -- millis() pengirim
                    check (sent_ms between 0 and 4294967295),
    waktu_diterima  text not null                    -- ingestion time (UTC)
                    check (waktu_diterima like '____-__-__T__:__:__%+00:00'),
    local_write_ms  real,                            -- durasi commit lokal
    synced_at       text,                            -- waktu salin Supabase
    sync_request_ms real                             -- durasi unggah batch
);
create index if not exists readings_received_idx
    on readings(waktu_diterima desc);
create index if not exists readings_device_time_idx
    on readings(device_id, waktu_diterima desc);
-- Kunci idempotensi: pengiriman ulang tidak membuat record ganda.
create unique index if not exists readings_identity_idx
    on readings(device_id, session_id, protokol, seq)
    where session_id is not null;
```

Gambar 3. Kode CREATE TABLE `devices`, `readings`, dan indeksnya (database/schema.sql baris 1–51)

Penjelasan Gambar 3:

- **Baris 9–18, tabel devices.** `device_id` adalah PRIMARY KEY NOT NULL dengan CHECK panjang 1–32 karakter, sama dengan aturan validasi payload. `nama` wajib diisi, `lokasi` opsional. `status` berisi default `'aktif'` dengan CHECK `aktif/nonaktif`; perangkat nonaktif akan ditolak saat ingestion. `pertama_terlihat` dan `terakhir_terlihat` diisi otomatis oleh server.
- **Baris 21–43, tabel readings.** `event_id` (UUID dari server) menjadi PRIMARY KEY. `device_id` memakai `REFERENCES devices(device_id)`, sehingga pembacaan dari perangkat yang tidak terdaftar ditolak basis data. Setiap kolom nilai diberi CHECK sesuai rentang sensor DHT22: suhu −40 sampai 125 °C, kelembapan 0 sampai 100 %RH. Protokol hanya boleh MQTT atau HTTP, `source_mode` hanya dht22/random/demo/unknown, dan `seq`/`sent_ms` berada pada rentang bilangan 32-bit. Kolom `local_write_ms`, `synced_at`, dan `sync_request_ms` menyimpan metadata proses.
- **Baris 44–51, indeks.** `readings_received_idx` mempercepat pengambilan data terbaru. `readings_device_time_idx` mengikuti saran materi (indeks `device_id`, `timestamp`). `readings_identity_idx` adalah indeks UNIQUE pada `(device_id, session_id, protokol, seq)` sebagai kunci idempotensi, sehingga pengiriman ulang tidak menghasilkan record ganda. Klausa `WHERE session_id IS NOT NULL` membuat data lama tanpa sesi tetap dapat disimpan.



Gambar 4. Kode CREATE TABLE `delivery_attempts`, `alerts`, dan data awal perangkat (baris 53–101)

Penjelasan Gambar 4:

- **Baris 54–75, tabel delivery_attempts.** Tabel ini menyimpan laporan RTT dari pengirim. `event_id` berupa UUID v5 yang dihitung dari identitas pengiriman. `status` dibatasi `ok/timeout/error`. Constraint tingkat tabel mewajibkan `rtt_ms` terisi (0–120.000 ms) jika status `ok` dan kosong jika gagal, sehingga kegagalan tidak dihitung sebagai RTT 0. UNIQUE pada identitas mencegah laporan ganda.
- **Baris 78–93, tabel alerts.** `alert_id` INTEGER PRIMARY KEY AUTOINCREMENT. `event_id` merujuk ke readings dengan `ON DELETE CASCADE`, sehingga alarm ikut terhapus saat

data mentah dihapus pada proses retensi. `device_id` merujuk ke devices. `parameter` dan `severity` dibatasi CHECK, dan `UNIQUE (event_id, parameter)` memastikan maksimal satu alarm per parameter untuk setiap pembacaan.

- **Baris 94–101.** Indeks `alerts_device_time_idx` dipakai untuk menampilkan alarm terbaru per perangkat. `INSERT OR IGNORE` mengisi data awal dua perangkat yang dikenal tanpa menimpa perubahan manual; perangkat lain akan didaftarkan otomatis.

3.2 Kode Ingestion yang Mengisi Tabel

server.py · baris 192–228

```python
def register_devices(conn):
    """Register every device found in readings; manual name/location/status stay."""
    conn.execute("""insert into devices (device_id, nama, pertama_terlihat, terakhir_terlihat)
        select device_id, device_id, min(waktu_diterima), max(waktu_diterima)
        from readings where true group by device_id """ + DEVICE_SEEN_UPSERT)


def record_alerts(conn, event_id=None):
    """Store the most severe alert per parameter for one reading, or for all readings."""
    scope, scope_args = ("and event_id = ?", (event_id,)) if event_id else ("", ())
    for parameter, operator, threshold, severity, message in ALERT_RULES:
        # Column and operator come from the constant rules above, never from a payload.
        conn.execute(f"""insert into alerts
            (event_id, device_id, waktu, parameter, nilai, ambang, severity, pesan)
            select event_id, device_id, waktu_diterima, ?, {parameter}, ?, ?, ?
            from readings where {parameter} {operator} ? {scope}
            on conflict(event_id, parameter) do nothing""",
            (parameter, threshold, severity, message, threshold, *scope_args))


def init_db(path):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    # Foreign keys stay off while an older log is rebuilt inside one transaction;
    # legacy_alter_table keeps references to "readings" pointing at the new table.
    with connect_db(path, foreign_keys=False) as conn:
        conn.execute("pragma journal_mode=WAL")
        conn.execute("pragma legacy_alter_table=on")
        conn.execute("begin immediate")
        tables = {row["name"] for row in conn.execute(
            "select name from sqlite_master where type = 'table'")}
        upgrade_tables(conn, tables)
        apply_schema(conn)
        register_devices(conn)
        if "alerts" not in tables:
            record_alerts(conn)  # Evaluate existing readings once, when alerts is new.
        if conn.execute("pragma foreign_key_check").fetchall():
            raise sqlite3.IntegrityError("Relasi tabel basis data tidak konsisten")
```

Gambar 5. Fungsi `register_devices()`, `record_alerts()`, dan `init_db()` pada server.py

- **init_db()** mengaktifkan mode WAL, lalu dalam satu transaksi memperbarui tabel versi UTS dan menjalankan `schema.sql`. Tabel lama yang belum memiliki FK/CHECK dibangun ulang tanpa kehilangan data. Fungsi ini juga mendaftarkan perangkat dari data lama, menghitung alarm historis satu kali, dan menutup proses dengan `PRAGMA foreign_key_check` untuk memastikan tidak ada relasi yatim. Pada database UTS, 2.263 pembacaan dan 1.964 laporan RTT tetap utuh (jumlah dan total nilainya sama sebelum dan sesudah), 1.609 alarm historis terbentuk, dan `PRAGMA integrity_check` bernilai `ok`.
- **register_devices()** memakai `INSERT … SELECT … GROUP BY device_id` dengan UPSERT (`ON CONFLICT DO UPDATE`). Hasilnya, waktu pertama dan terakhir terlihat diperbarui tanpa

menimpa nama, lokasi, atau status yang diisi manual.

- **record_alerts()** menjalankan `INSERT … SELECT` untuk setiap aturan ambang: suhu ≥ 32 °C (kritis), suhu ≥ 30 °C (peringatan), kelembapan ≥ 80 %RH (peringatan), dan kelembapan ≤ 50 %RH (peringatan). Aturan terparah dievaluasi lebih dulu, dan `ON CONFLICT(event_id, parameter) DO NOTHING` membuat hanya alarm terparah per parameter yang tersimpan. Ambang ini adalah asumsi demo dan dapat diubah sesuai kebutuhan ruangan.

server.py — server.py · baris 231–276

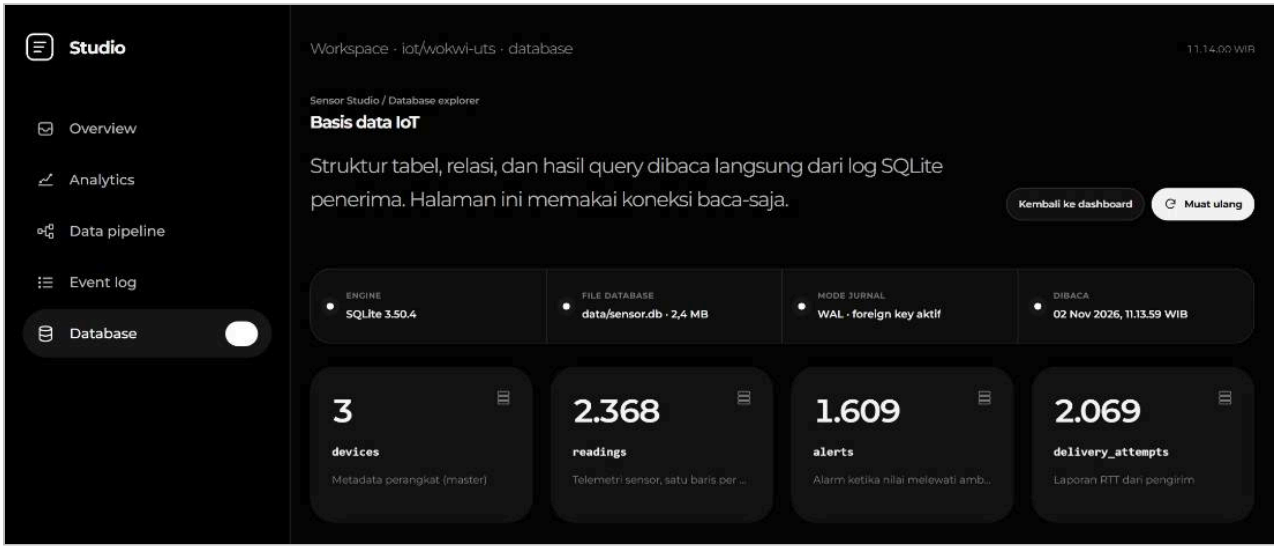
```
231 def save_reading(db_path, payload, protocol, received_at):
232     started = time.perf_counter()
233     reading = validate_payload(payload)
234     event_id = str(uuid.uuid4())
235     with connect_db(db_path) as conn:
236         conn.execute("begin immediate")
237         if reading["session_id"] is not None:
238             metric = conn.execute("""select source_mode from delivery_attempts where
239                 device_id=? and session_id=? and protokol=? and seq=?""",
240                 (reading["device_id"], reading["session_id"], protocol, reading["seq"])).fetchone()
241             if metric and metric["source_mode"] != reading["source_mode"]:
242                 raise PayloadConflict("Sumber pembacaan berbeda dari laporan metrik")
243             previous = conn.execute("""select * from readings where
244                 device_id=? and session_id=? and protokol=? and seq=?""",
245                 (reading["device_id"], reading["session_id"], protocol, reading["seq"])).fetchone()
246             if previous:
247                 if any(previous[field] != reading[field] for field in
248                        ("suhu", "kelembapan", "sent_ms", "source_mode")):
249                     raise PayloadConflict("Identitas pembacaan sudah dipakai untuk data berbeda")
250                 return dict(previous)
251         device = conn.execute("select status from devices where device_id=?",
252                               (reading["device_id"],)).fetchone()
253         if device and device["status"] != "aktif":
254             raise DeviceDisabled("Perangkat dinonaktifkan pada tabel devices")
255         # Unknown devices are registered in the same transaction (readings FK).
256         conn.execute("""insert into devices
257             (device_id, nama, pertama_terlihat, terakhir_terlihat) values (?, ?, ?, ?)
258             """ + DEVICE_SEEN_UPSERT,
259             (reading["device_id"], reading["device_id"], received_at, received_at))
260         conn.execute("""insert into readings
261             (event_id, device_id, seq, suhu, kelembapan, sent_ms,
262              protokol, waktu_diterima, session_id, source_mode)
263             values (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
264             (event_id, reading["device_id"], reading["seq"], reading["suhu"],
265              reading["kelembapan"], reading["sent_ms"], protocol, received_at,
266              reading["session_id"], reading["source_mode"]))
267         record_alerts(conn, event_id)
268         conn.commit()
269         local_ms = (time.perf_counter() - started) * 1000
270         conn.execute("update readings set local_write_ms=? where event_id=?",
271                      (local_ms, event_id))
272     live_state.changed()
273     log.info("DITERIMA %s %s seq=%d suhu=%.1f kelembapan=%.1f waktu=%s",
274              protocol, reading["device_id"], reading["seq"], reading["suhu"],
275              reading["kelembapan"], received_at)
276     return {**reading, "event_id": event_id, "waktu_diterima": received_at}
```

Gambar 6. Fungsi `save_reading()`: validasi, idempotensi, pendaftaran perangkat, INSERT, dan alarm

Gambar 6 menunjukkan jalur penyimpanan yang dipakai bersama oleh MQTT dan HTTP. Payload divalidasi terlebih dahulu dengan `validate_payload()`. Transaksi `BEGIN IMMEDIATE` lalu memeriksa identitas: retry identik mengembalikan record lama, sedangkan identitas sama dengan isi berbeda ditolak sebagai konflik (HTTP 409). Setelah itu status perangkat diperiksa; perangkat nonaktif ditolak (HTTP 403, MQTT tanpa ACK). Perangkat didaftarkan atau diperbarui dengan UPSERT, pembacaan disimpan dengan parameter terikat (`?`) sehingga nilai payload tidak pernah dirangkai menjadi SQL, dan alarm dibuat dengan `record_alerts()`. Durasi hingga commit dicatat pada `local_write_ms`.

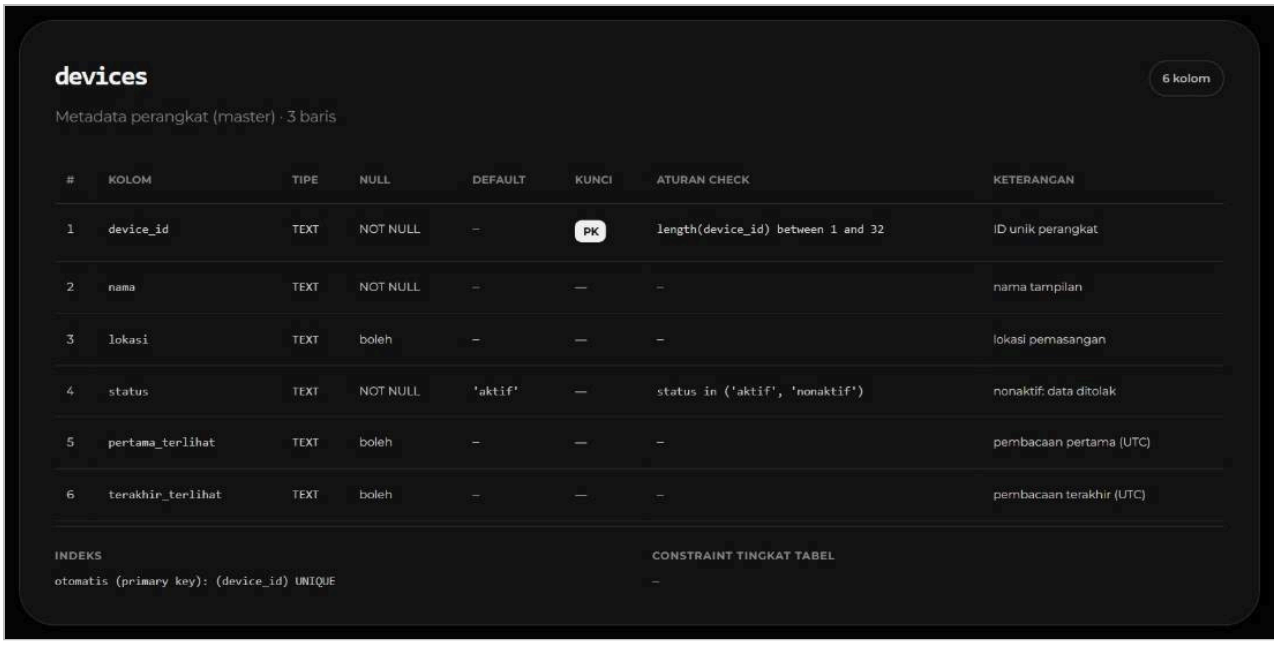
3.3 Struktur Tabel

Struktur tabel ditampilkan oleh halaman baru **Database Explorer** (`/database`) pada dashboard. Isinya dibaca langsung dari skema yang aktif melalui `PRAGMA table_info`, `foreign_key_list`, `index_list`, dan teks DDL di `sqlite_master`, bukan ditulis manual. Koneksi halaman ini bersifat baca-saja (`PRAGMA query_only`).



Gambar 7. Halaman Database Explorer: informasi basis data dan jumlah baris setiap tabel

Gambar 7 menunjukkan engine SQLite 3.50.4, file `data/sensor.db` berukuran sekitar 2,4 MB, dan mode jurnal WAL dengan foreign key aktif. Jumlah barisnya adalah devices 3, readings 2.368, alerts 1.609, dan delivery_attempts 2.069. Tabel devices berisi dua perangkat awal dan `esp32-02` yang sengaja didaftarkan berstatus nonaktif untuk uji validasi identitas.



devices

6 kolom

Metadata perangkat (master) · 3 baris

#	KOLOM	TIPE	NULL	DEFAULT	KUNCI	ATURAN CHECK	KETERANGAN
1	device_id	TEXT	NOT NULL	–	PK	length(device_id) between 1 and 32	ID unik perangkat
2	nama	TEXT	NOT NULL	–	—	–	nama tampilan
3	lokasi	TEXT	boleh	–	—	–	lokasi pemasangan
4	status	TEXT	NOT NULL	'aktif'	—	status in ('aktif', 'nonaktif')	nonaktif: data ditolak
5	pertama_terlihat	TEXT	boleh	–	—	–	pembacaan pertama (UTC)
6	terakhir_terlihat	TEXT	boleh	–	—	–	pembacaan terakhir (UTC)

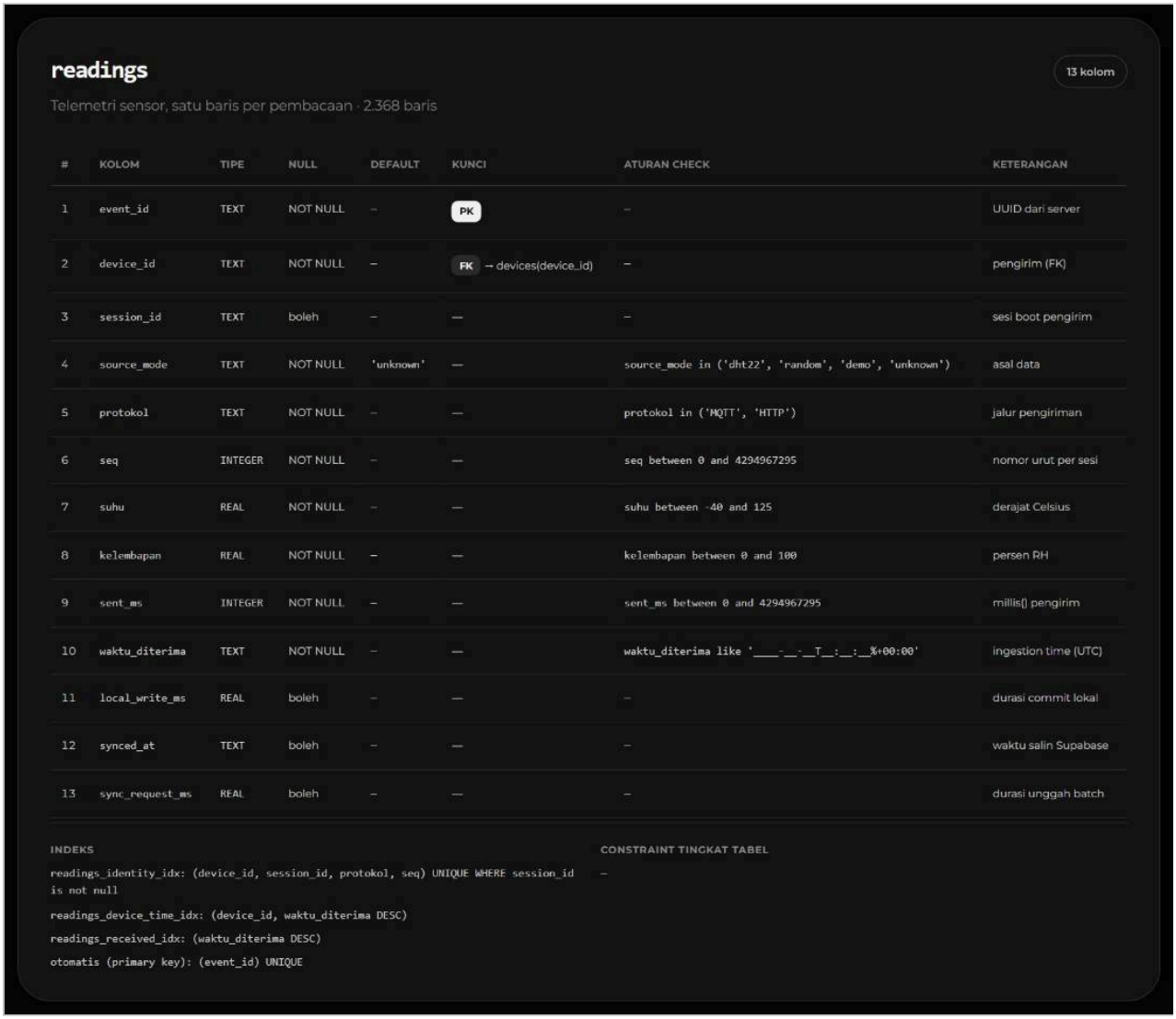
INDEKS

otomatis (primary key): (device_id) UNIQUE

CONSTRAINT TINGKAT TABEL

–

Gambar 8. Struktur tabel `devices`



readings

13 kolom

Telemetri sensor, satu baris per pembacaan · 2.368 baris

#	KOLOM	TIPE	NULL	DEFAULT	KUNCI	ATURAN CHECK	KETERANGAN
1	event_id	TEXT	NOT NULL	–	PK	–	UUID dari server
2	device_id	TEXT	NOT NULL	–	FK → devices(device_id)	–	pengirim (FK)
3	session_id	TEXT	boleh	–	—	–	sesi boot pengirim
4	source_mode	TEXT	NOT NULL	'unknown'	—	source_mode in ('dht22', 'random', 'demo', 'unknown')	asal data
5	protokol	TEXT	NOT NULL	–	—	protokol in ('MQTT', 'HTTP')	jalur pengiriman
6	seq	INTEGER	NOT NULL	–	—	seq between 0 and 4294967295	nomor urut per sesi
7	suhu	REAL	NOT NULL	–	—	suhu between -40 and 125	derajat Celsius
8	kelembapan	REAL	NOT NULL	–	—	kelembapan between 0 and 100	persen RH
9	sent_ms	INTEGER	NOT NULL	–	—	sent_ms between 0 and 4294967295	millis() pengirim
10	waktu_diterima	TEXT	NOT NULL	–	—	waktu_diterima like '____-__-__T__:__:__%+00:00'	ingestion time (UTC)
11	local_write_ms	REAL	boleh	–	—	–	durasi commit lokal
12	synced_at	TEXT	boleh	–	—	–	waktu salin Supabase
13	sync_request_ms	REAL	boleh	–	—	–	durasi unggah batch

INDEKS

readings_identity_idx: (device_id, session_id, protokol, seq) UNIQUE WHERE session_id is not null

readings_device_time_idx: (device_id, waktu_diterima DESC)

readings_received_idx: (waktu_diterima DESC)

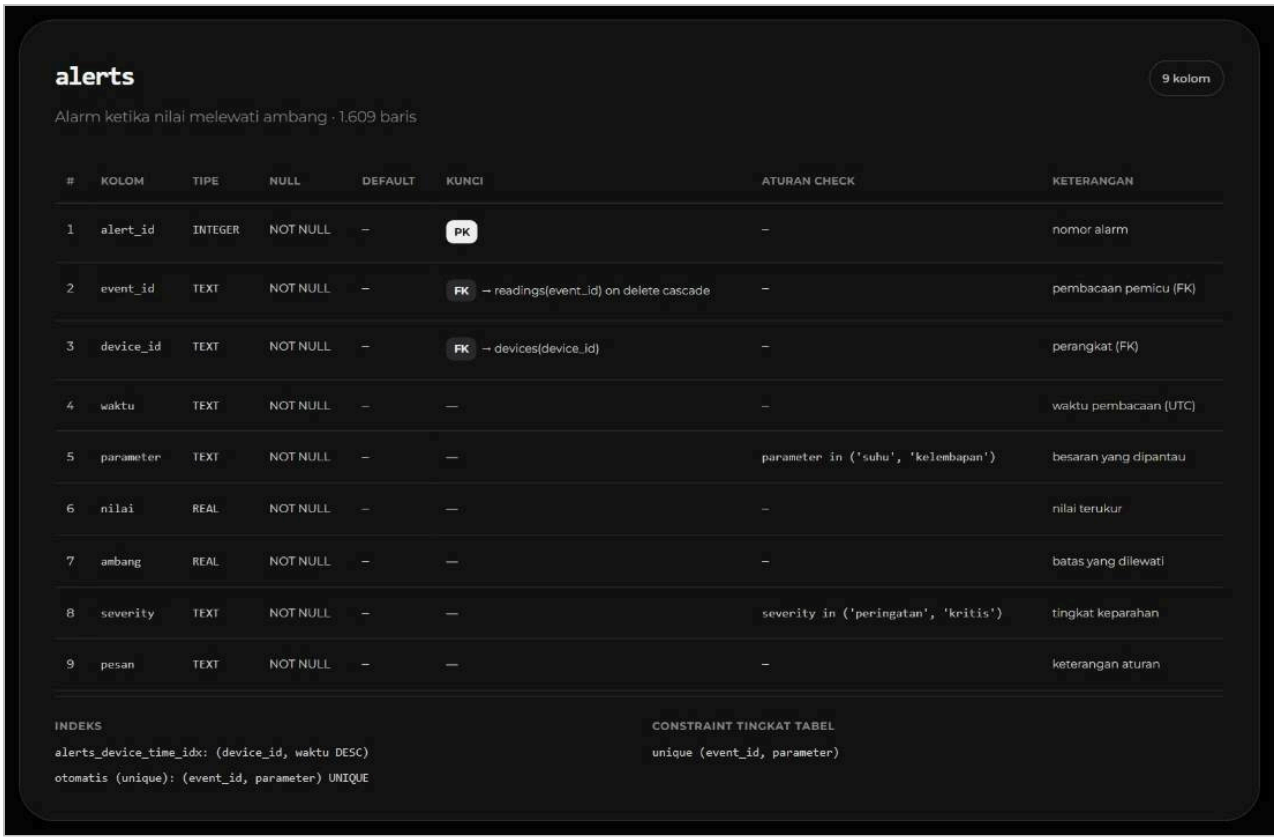
otomatis (primary key): (event_id) UNIQUE

CONSTRAINT TINGKAT TABEL

–

Gambar 9. Struktur tabel `readings`

Kolom pada Gambar 8 dan 9 berarti: **TIPE** adalah kelas penyimpanan SQLite (TEXT, INTEGER, REAL); **NULL** menunjukkan apakah kolom wajib diisi; **KUNCI** menandai PK atau FK beserta tabel rujukannya; **ATURAN CHECK** adalah aturan validasi di tingkat basis data; dan **KETERANGAN** diambil dari komentar pada schema.sql. Tabel devices memiliki 6 kolom dengan PK `device_id` dan default status `'aktif'`. Tabel readings memiliki 13 kolom: PK `event_id`, FK `device_id → devices(device_id)`, tujuh aturan CHECK, dan empat indeks, termasuk indeks unik parsial `readings_identity_idx`. Timestamp disimpan sebagai TEXT ISO 8601 karena SQLite tidak memiliki tipe tanggal khusus.



alerts

9 kolom

Alarm ketika nilai melewati ambang · 1.609 baris

#	KOLOM	TIPE	NULL	DEFAULT	KUNCI	ATURAN CHECK	KETERANGAN
1	alert_id	INTEGER	NOT NULL	–	PK	–	nomor alarm
2	event_id	TEXT	NOT NULL	–	FK → readings(event_id) on delete cascade	–	pembacaan pemicu (FK)
3	device_id	TEXT	NOT NULL	–	FK → devices(device_id)	–	perangkat (FK)
4	waktu	TEXT	NOT NULL	–	—	–	waktu pembacaan (UTC)
5	parameter	TEXT	NOT NULL	–	—	parameter in ('suhu', 'kelembapan')	besaran yang dipantau
6	nilai	REAL	NOT NULL	–	—	–	nilai terukur
7	ambang	REAL	NOT NULL	–	—	–	batas yang dilewati
8	severity	TEXT	NOT NULL	–	—	severity in ('peringatan', 'kritis')	tingkat keparahan
9	pesan	TEXT	NOT NULL	–	—	–	keterangan aturan

INDEKS

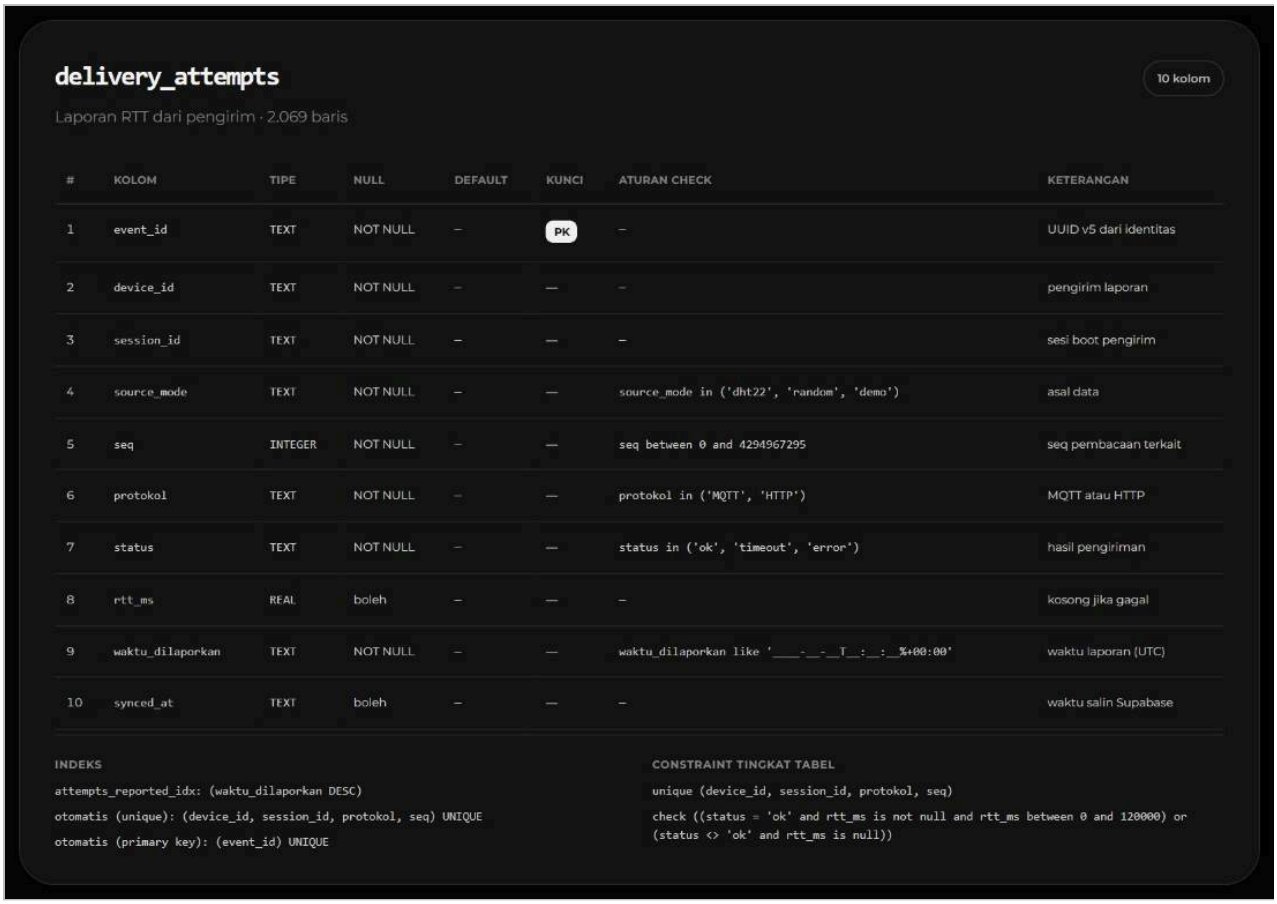
alerts_device_time_idx: (device_id, waktu DESC)

otomatis (unique): (event_id, parameter) UNIQUE

CONSTRAINT TINGKAT TABEL

unique (event_id, parameter)

Gambar 10. Struktur tabel `alerts`



delivery_attempts

10 kolom

Laporan RTT dari pengirim · 2.069 baris

#	KOLOM	TIPE	NULL	DEFAULT	KUNCI	ATURAN CHECK	KETERANGAN
1	event_id	TEXT	NOT NULL	–	PK	–	UUID v5 dari identitas
2	device_id	TEXT	NOT NULL	–	—	–	pengirim laporan
3	session_id	TEXT	NOT NULL	–	—	–	sesi boot pengirim
4	source_mode	TEXT	NOT NULL	–	—	source_mode in ('dht22', 'random', 'demo')	asal data
5	seq	INTEGER	NOT NULL	–	—	seq between 0 and 4294967295	seq pembacaan terkait
6	protokol	TEXT	NOT NULL	–	—	protokol in ('MQTT', 'HTTP')	MQTT atau HTTP
7	status	TEXT	NOT NULL	–	—	status in ('ok', 'timeout', 'error')	hasil pengiriman
8	rtt_ms	REAL	boleh	–	—	–	kosong jika gagal
9	waktu_dilaporkan	TEXT	NOT NULL	–	—	waktu_dilaporkan like '____-__-__T__:__:__%+00:00'	waktu laporan (UTC)
10	synced_at	TEXT	boleh	–	—	–	waktu salin Supabase

INDEKS

attempts_reported_idx: (waktu_dilaporkan DESC)

otomatis (unique): (device_id, session_id, protokol, seq) UNIQUE

otomatis (primary key): (event_id) UNIQUE

CONSTRAINT TINGKAT TABEL

unique (device_id, session_id, protokol, seq)

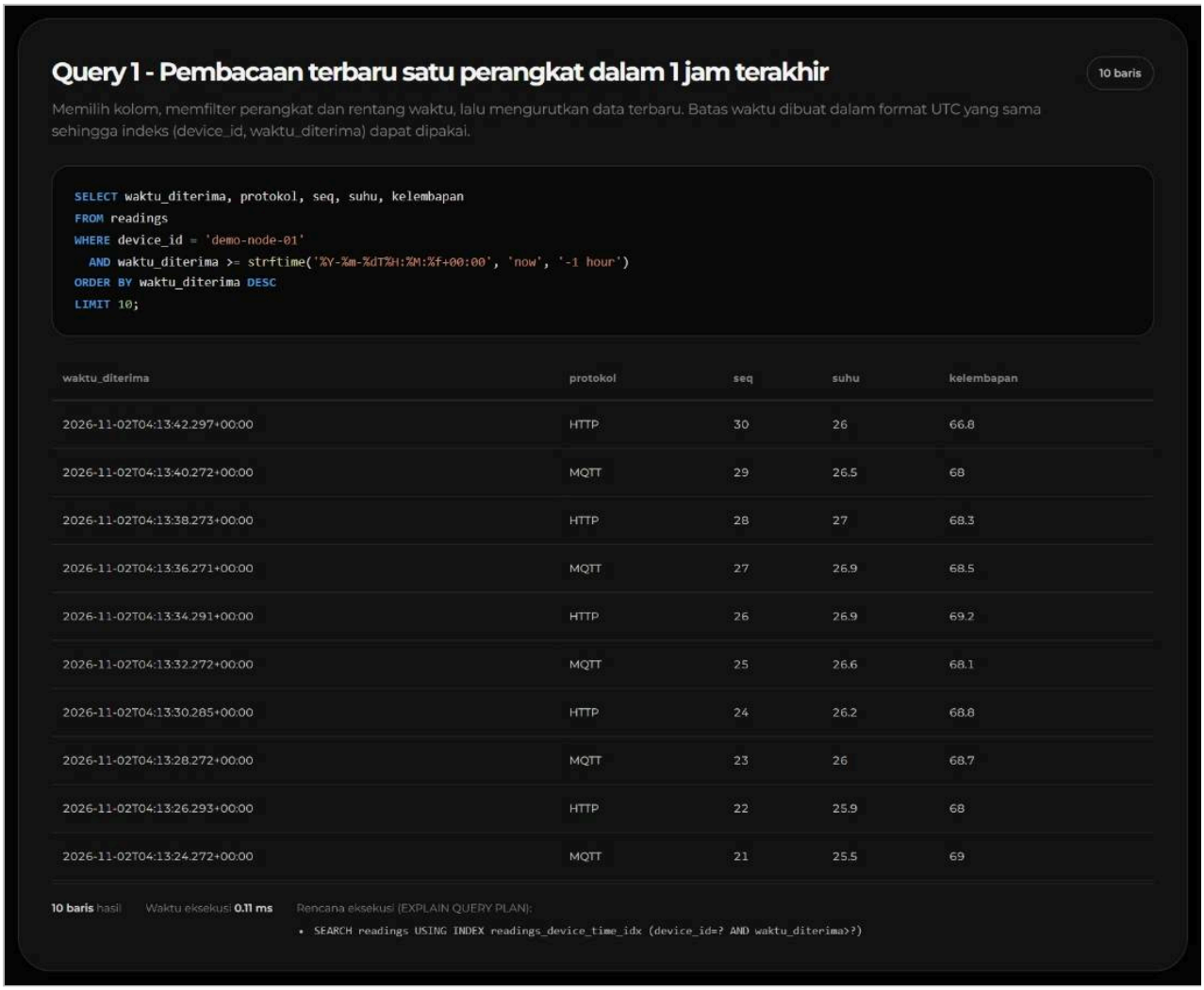
check ((status = 'ok' and rtt_ms is not null and rtt_ms between 0 and 120000) or (status <> 'ok' and rtt_ms is null))

Gambar 11. Struktur tabel `delivery_attempts`

Gambar 10 memperlihatkan dua foreign key pada alerts: `event_id → readings(event_id) on delete cascade` dan `device_id → devices(device_id)`, serta constraint tingkat tabel `unique (event_id, parameter)`. Gambar 11 memperlihatkan delivery_attempts dengan 10 kolom, constraint UNIQUE pada identitas pengiriman, dan CHECK yang menghubungkan `status` dengan `rtt_ms`. Bagian INDEKS menunjukkan indeks buatan sendiri dan indeks otomatis yang dibuat SQLite untuk PRIMARY KEY dan UNIQUE.

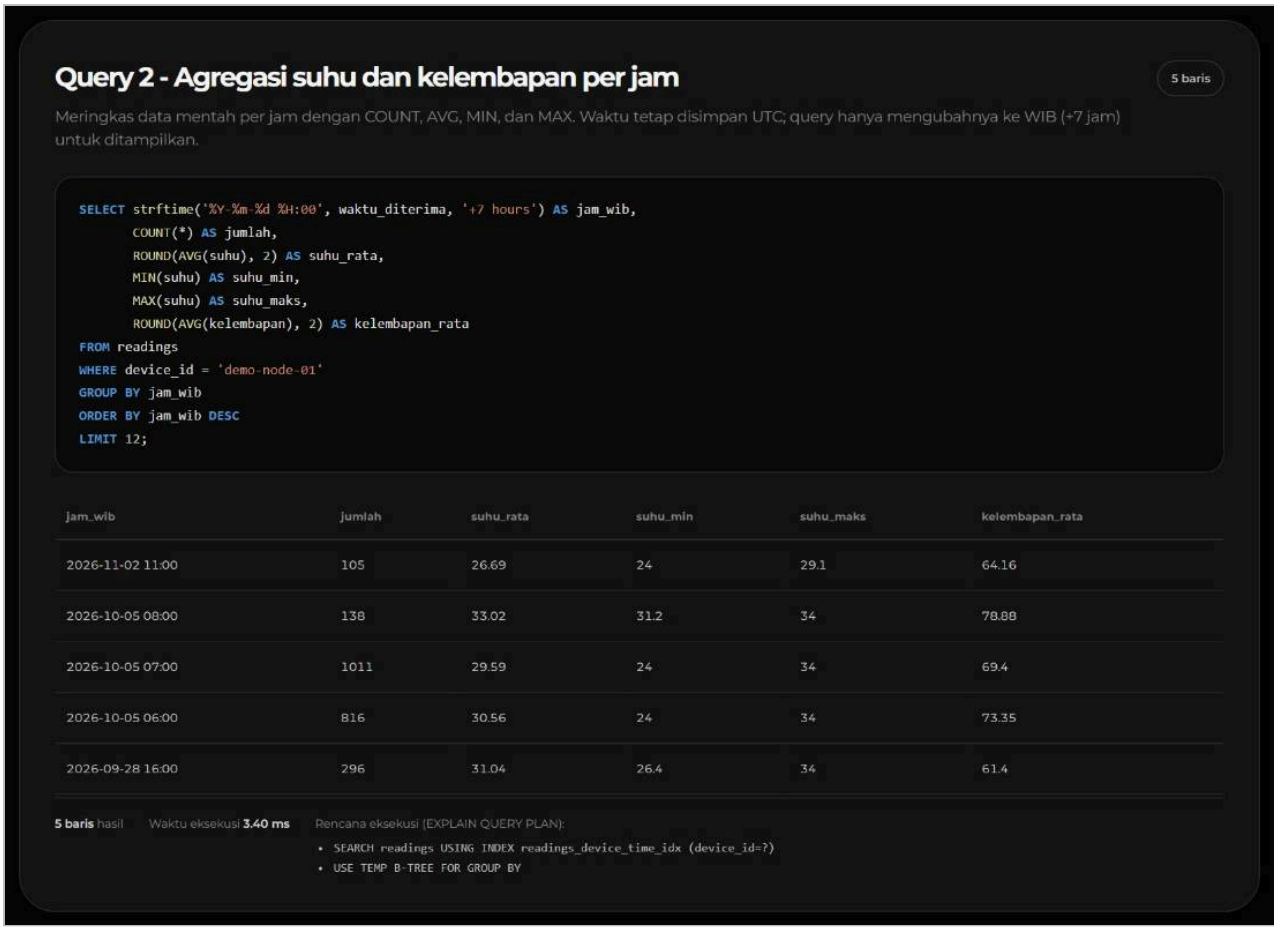
3.4 Query dan Hasilnya

Query disimpan di `database/queries.sql` dan dijalankan oleh halaman Database Explorer pada 2 November 2026 pukul 11.14 WIB, setelah node demo berhenti. Setiap gambar memuat SQL, hasil, waktu eksekusi, dan rencana eksekusi (`EXPLAIN QUERY PLAN`). Karena memakai SQLite, fungsi PostgreSQL pada materi (`date_trunc`, `NOW() - INTERVAL`) diganti dengan `strftime()`.



Gambar 12. Query 1 – pembacaan terbaru satu perangkat dalam 1 jam terakhir

Query 1 memilih kolom, memfilter perangkat `demo-node-01` dan rentang 1 jam terakhir, lalu mengurutkan dari yang terbaru (materi slide 14). Batas waktu dibuat dengan format UTC yang sama dengan kolom, sehingga perbandingan tetap memakai indeks. Hasilnya 10 baris terbaru. Baris teratas diterima pukul `04:13:42.297` UTC (11.13.42 WIB) dengan seq 30, HTTP, 26 °C, dan 66,8 %RH. Nilai `seq` bergantian MQTT (ganjil) dan HTTP (genap), membuktikan kedua protokol tersimpan di tabel yang sama. Rencana eksekusi `SEARCH readings USING INDEX readings_device_time_idx (device_id=? AND waktu_diterima>?)` menunjukkan indeks benar-benar dipakai, sehingga query selesai dalam kurang dari 1 ms.

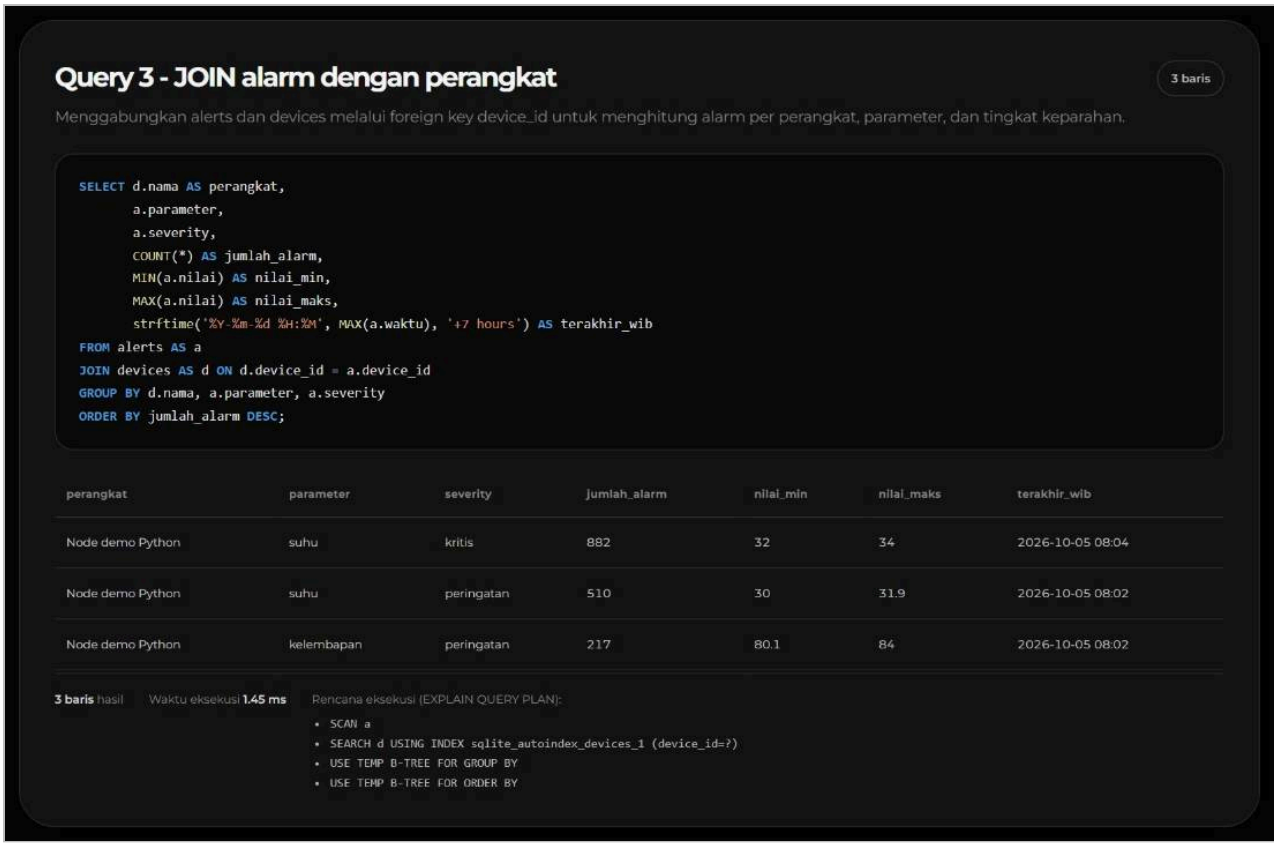


Gambar 13. Query 2 – agregasi suhu dan kelembapan per jam

Query 2 meringkas data mentah per jam dengan `COUNT`, `AVG`, `MIN`, dan `MAX` (materi slide 15). Waktu disimpan UTC dan dikonversi ke WIB dengan `strftime(…, '+7 hours')` hanya untuk tampilan. Sebanyak 2.366 baris milik `demo-node-01` diringkas menjadi 5 baris:

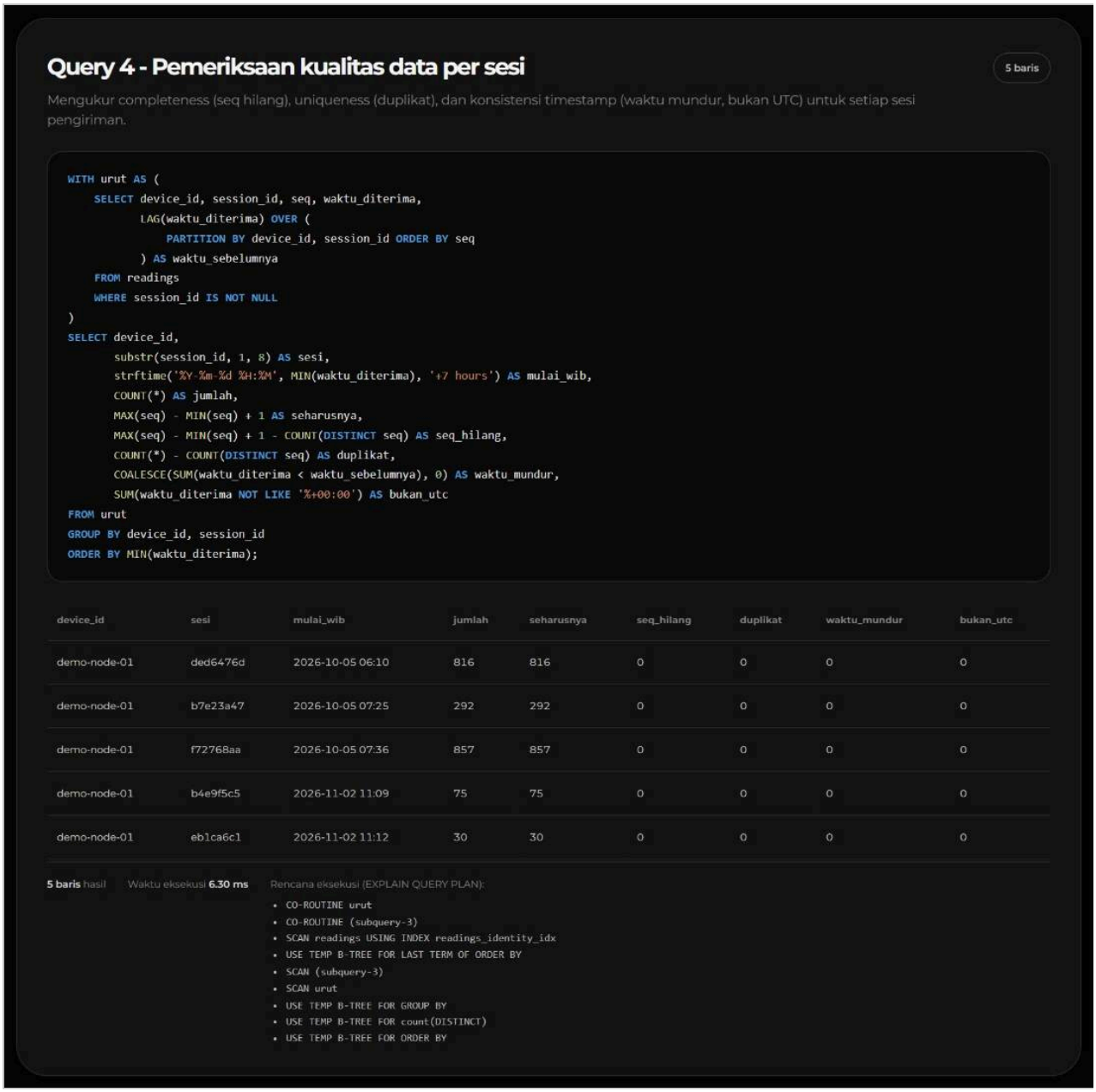
Jam (WIB)	Jumlah	Suhu rata-rata	Suhu min–maks	Kelembapan rata-rata
2026-11-02 11:00	105	26,69 °C	24,0–29,1 °C	64,16 %RH
2026-10-05 08:00	138	33,02 °C	31,2–34,0 °C	78,88 %RH
2026-10-05 07:00	1.011	29,59 °C	24,0–34,0 °C	69,40 %RH
2026-10-05 06:00	816	30,56 °C	24,0–34,0 °C	73,35 %RH
2026-09-28 16:00	296	31,04 °C	26,4–34,0 °C	61,40 %RH

AVG menggambarkan kondisi umum setiap jam, sedangkan MAX menunjukkan nilai puncak. Jam 08.00 WIB pada 5 Oktober paling panas (rata-rata 33,02 °C), sedangkan sesi 2 November paling sejuk. Ringkasan seperti ini menjadi dasar *downsampling* untuk kebijakan retensi.



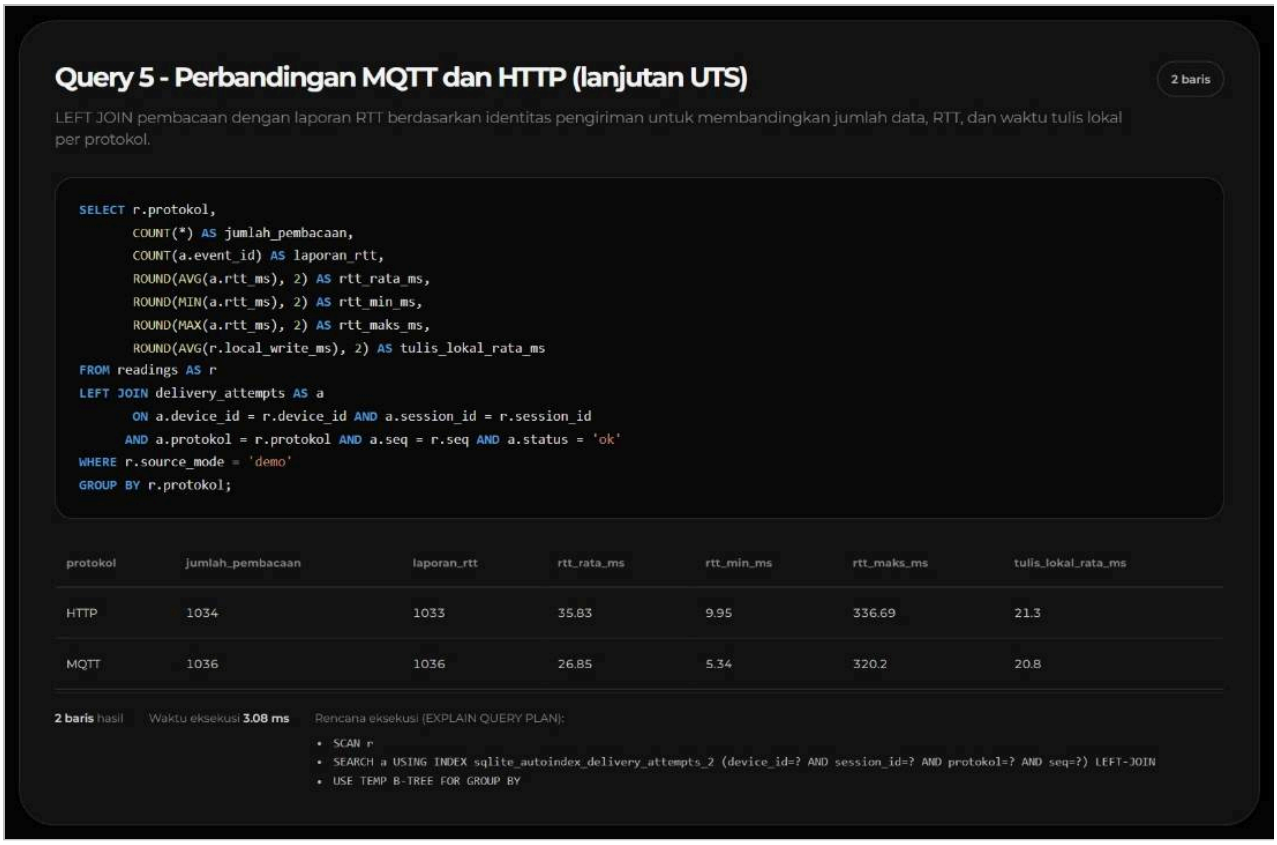
Gambar 14. Query 3 – JOIN tabel alerts dan devices

Query 3 menggabungkan `alerts` dengan `devices` melalui foreign key `device_id`, lalu mengelompokkan per perangkat, parameter, dan tingkat keparahan. Hasilnya tiga kelompok untuk Node demo Python: suhu kritis 882 alarm (32–34 °C), suhu peringatan 510 alarm (30–31,9 °C), dan kelembapan peringatan 217 alarm (80,1–84 %RH). Totalnya 1.609, sama dengan jumlah baris tabel alerts. Alarm terakhir terjadi pada 5 Oktober 2026 pukul 08.04 WIB. Data 2 November tidak memicu alarm karena suhu maksimumnya 29,1 °C, dan tidak ada pembacaan dengan kelembapan ≤ 50 %RH. Rencana eksekusi menunjukkan JOIN mencari perangkat melalui indeks primary key devices.



Gambar 15. Query 4 – pemeriksaan kualitas data per sesi

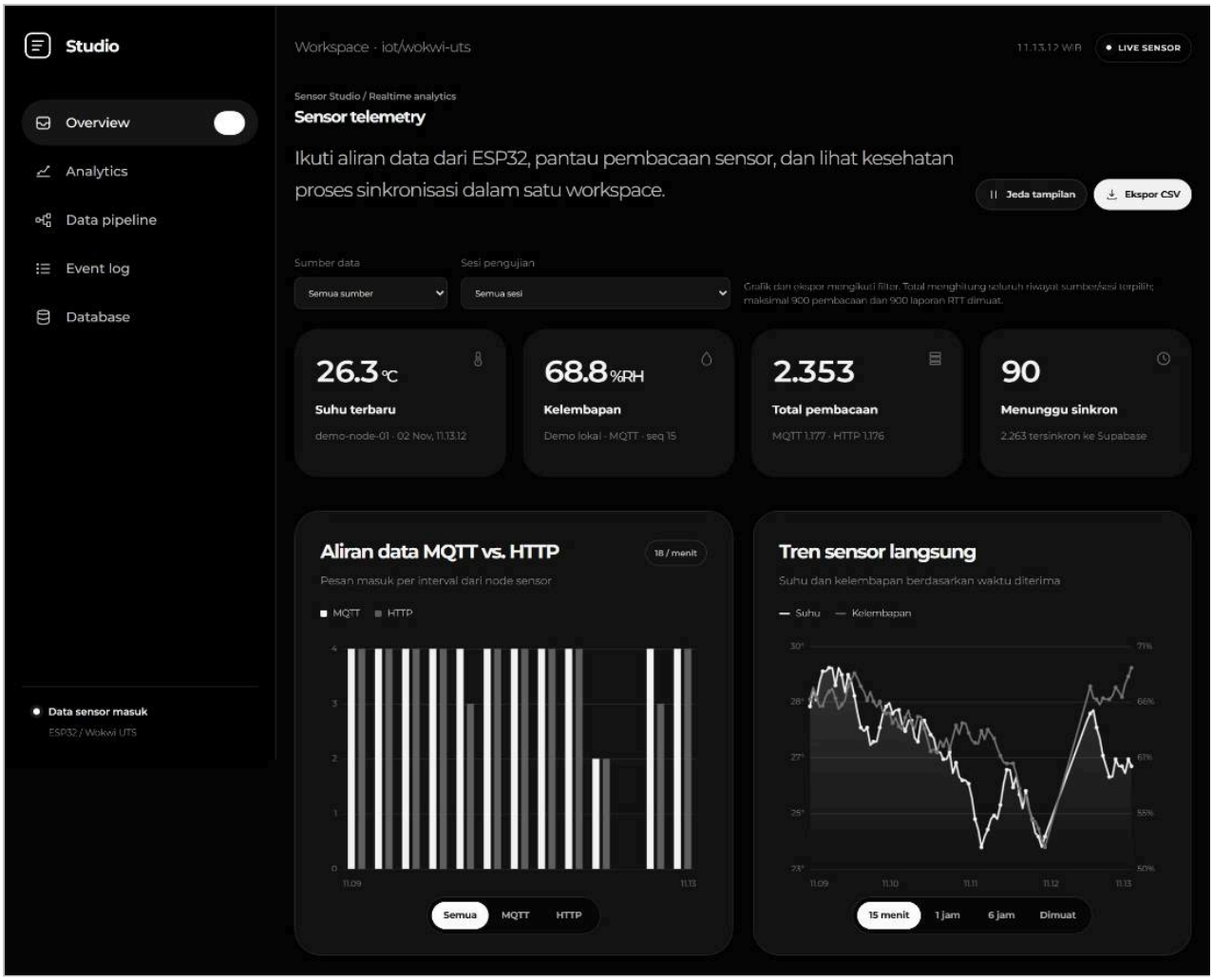
Query 4 mengukur dimensi kualitas data dari materi. Window function `LAG()` membandingkan waktu setiap record dengan record ber-`seq` sebelumnya. Kolom `seharusnya` dihitung dari `MAX(seq) - MIN(seq) + 1`. Hasilnya ada lima sesi: tiga dari 5 Oktober (816, 292, dan 857 record) dan dua dari 2 November (75 dan 30 record). Semua sesi memiliki `jumlah = seharusnya`, `seq_hilang = 0` (completeness), `duplikat = 0` (uniqueness), `waktu_mundur = 0`, dan `bukan_utc = 0` (konsistensi timestamp). Sebanyak 298 record format lama tanpa `session_id` tidak ikut dihitung karena tidak memiliki nomor urut per sesi.



Gambar 16. Query 5 – perbandingan MQTT dan HTTP (lanjutan UTS)

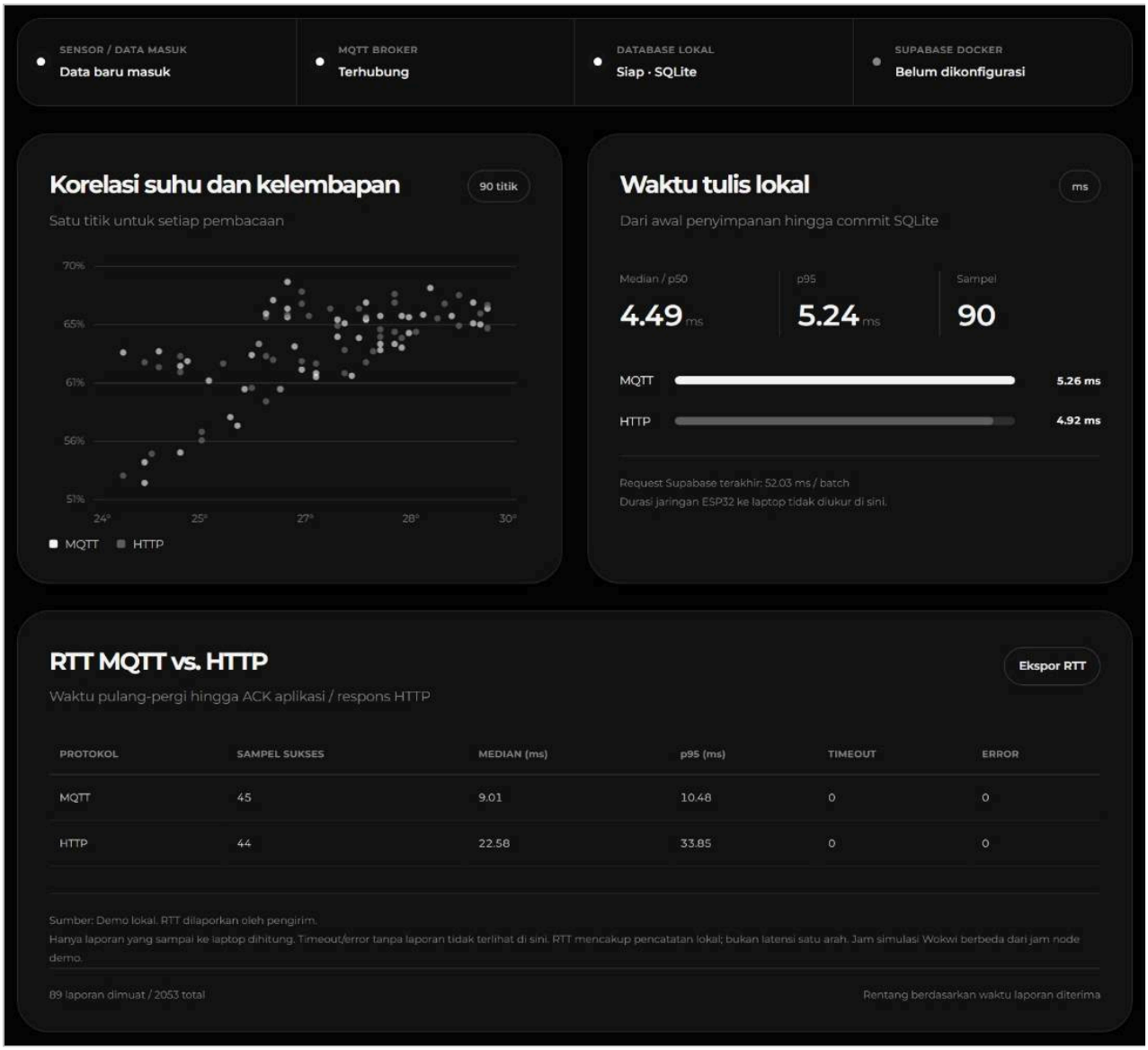
Query 5 melanjutkan topik UTS. Query ini memakai `LEFT JOIN` antara readings dan delivery_attempts berdasarkan identitas pengiriman. LEFT JOIN dipilih agar pembacaan tanpa laporan RTT tetap terhitung. HTTP memiliki 1.034 pembacaan dan 1.033 laporan RTT (satu laporan tidak sampai), dengan RTT rata-rata 35,83 ms (9,95–336,69 ms). MQTT memiliki 1.036 pembacaan dan 1.036 laporan, dengan RTT rata-rata 26,85 ms (5,34–320,2 ms). Waktu tulis lokal keduanya hampir sama (21,3 ms dan 20,8 ms). Pada data demo lokal ini MQTT lebih cepat, karena koneksinya dipertahankan sementara HTTP membuat request baru setiap pembacaan. Hasil ini belum mewakili jaringan ESP32 sebenarnya.

3.5 Dashboard



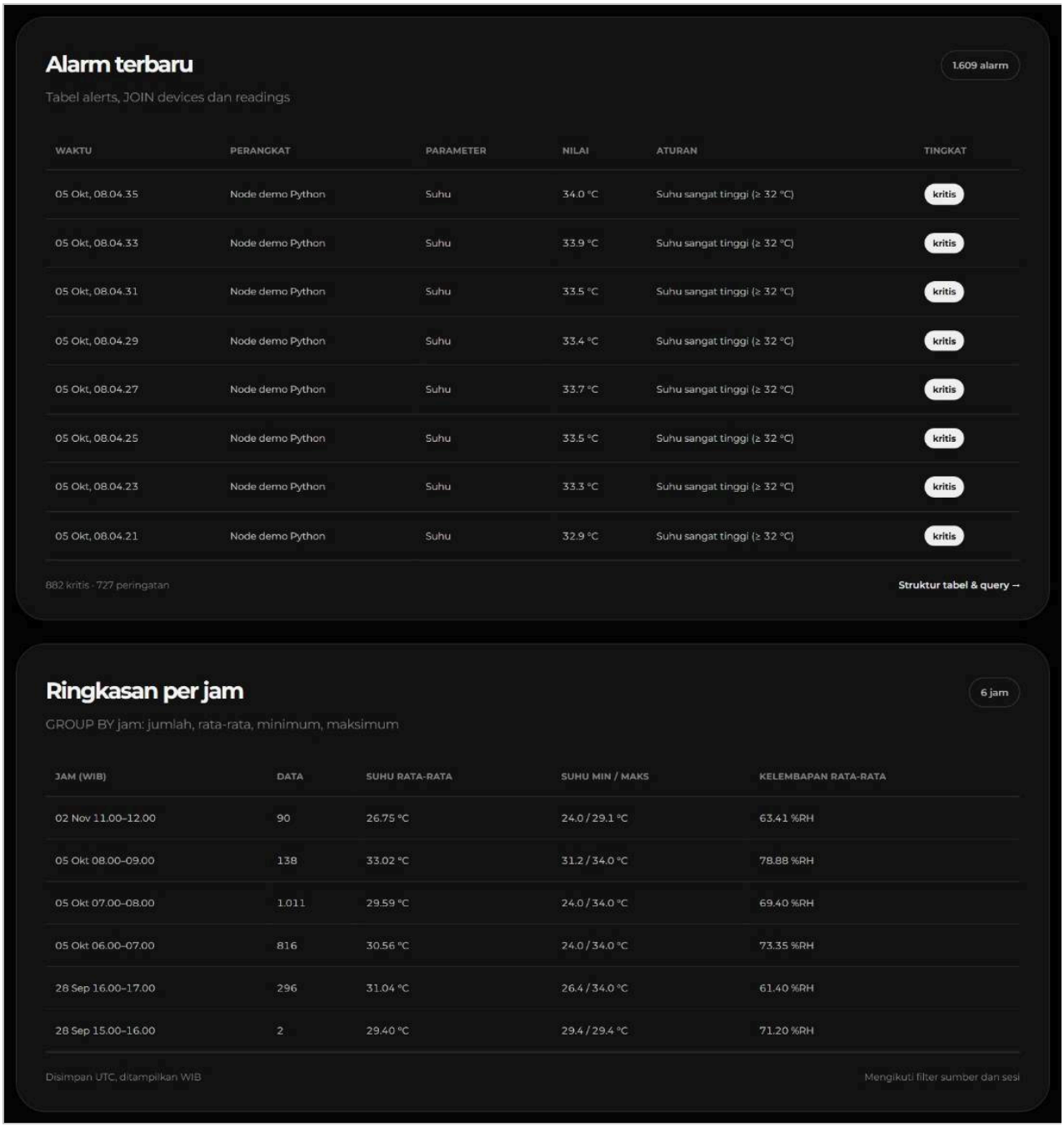
Gambar 17. Dashboard utama saat node sensor mengirim data (status LIVE SENSOR)

Gambar 17 diambil pukul 11.13.12 WIB ketika node demo sedang mengirim data, sehingga statusnya **LIVE SENSOR**. Kartu ringkasan menampilkan suhu terbaru 26,3 °C dan kelembapan 68,8 %RH (MQTT, seq 15), total 2.353 pembacaan (MQTT 1.177, HTTP 1.176), serta 90 record menunggu sinkronisasi. Jumlah ini sedikit lebih kecil daripada hasil query (2.368) karena node masih berjalan saat gambar diambil. Grafik kiri menghitung pesan MQTT dan HTTP per interval; grafik kanan menampilkan tren suhu dan kelembapan pada rentang 15 menit. Semua angka berasal dari endpoint `/api/dashboard` yang membaca tabel SQLite.



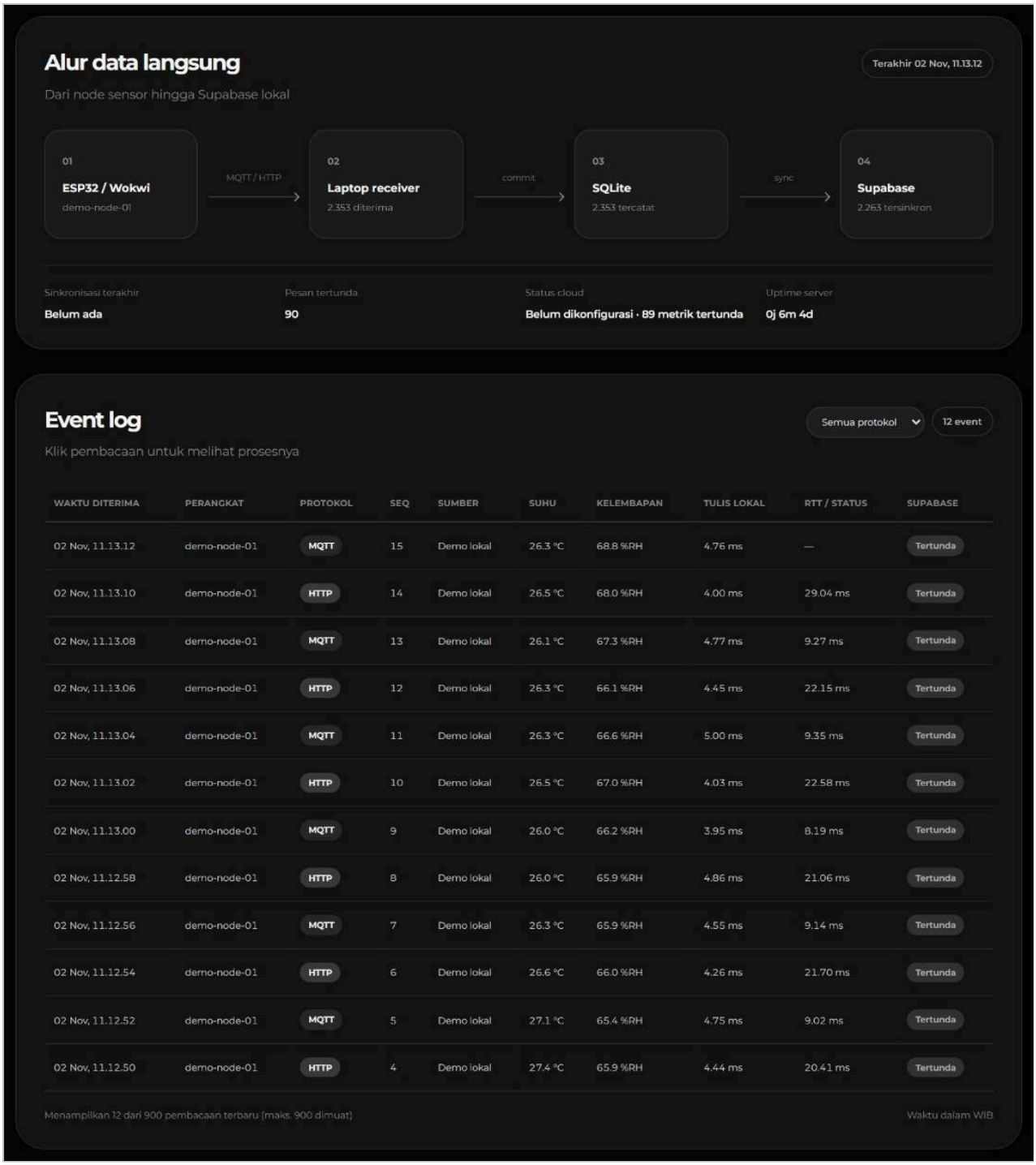
Gambar 18. Status layanan, korelasi, waktu tulis lokal, dan perbandingan RTT

Gambar 18 menunjukkan status layanan: data sensor masuk, broker MQTT terhubung, database lokal SQLite siap, dan Supabase belum dikonfigurasi (sengaja tidak dijalankan). Diagram sebar memuat 90 titik pembacaan pada rentang 15 menit. Waktu tulis lokal hingga commit memiliki median 4,49 ms dan p95 5,24 ms. Tabel RTT dari delivery_attempts menunjukkan median MQTT 9,01 ms (45 sampel) dan HTTP 22,58 ms (44 sampel) tanpa timeout maupun error.



Gambar 19. Panel baru di dashboard: Alarm terbaru dan Ringkasan per jam

Gambar 19 adalah panel baru hasil pengembangan tugas ini. **Alarm terbaru** menampilkan hasil JOIN alerts–devices–readings (total 1.609 alarm: 882 kritis dan 727 peringatan). **Ringkasan per jam** menampilkan agregasi GROUP BY per jam dengan logika yang sama seperti Query 2. Waktu disimpan UTC dan ditampilkan WIB. Kedua panel mengikuti filter sumber data dan sesi di bagian atas dashboard.



Gambar 20. Alur data langsung dan event log pembacaan terbaru

Gambar 20 memperlihatkan alur ESP32/node → receiver (2.353 diterima) → SQLite (2.353 tercatat) → Supabase (2.263 tersinkron dari pengujian sebelumnya). Karena Supabase tidak dijalankan, 90 pembacaan dan 89 laporan metrik tercatat sebagai tertunda; data tetap aman di SQLite dan akan diunggah saat layanan tersedia (prinsip *store and forward*). Event log menampilkan 12 pembacaan terbaru beserta protokol, seq, sumber, nilai sensor, durasi tulis lokal sekitar 4–5 ms, RTT, dan status sinkronisasi.

BAB IV
PENGUJIAN VALIDASI, TIMESTAMP, DAN KUALITAS DATA

4.1 Penolakan Payload Salah

Pengujian dijalankan dengan skrip `tugas/basis-data/tools/validasi.py` terhadap receiver yang sedang berjalan, melalui request HTTP dan pesan MQTT nyata. Jumlah record diperiksa sebelum dan sesudah setiap kasus.

No	Kasus uji	Diharapkan	Hasil	Pesan server
1	Suhu berupa teks	400	400	suhu harus angka terbatas
2	Suhu 200 °C (di luar rentang DHT22)	400	400	suhu di luar rentang DHT22
3	Kelembapan di atas 100 %	400	400	kelembapan di luar rentang DHT22
4	Field suhu hilang	400	400	Field sensor tidak lengkap atau tidak dikenal
5	Field asing ditambahkan	400	400	Field sensor tidak lengkap atau tidak dikenal
6	`device_id` mengandung spasi	400	400	device_id tidak valid
7	`seq` negatif	400	400	seq harus bilangan bulat 0..4294967295
8	`source_mode` tidak dikenal	400	400	source_mode harus dht22, random, atau demo
9	JSON rusak	400	400	Body harus objek JSON
10	Content-Type bukan JSON	415	415	Content-Type harus application/json
11	Perangkat nonaktif (`esp32-02`)	403	403	Perangkat dinonaktifkan pada tabel devices
12	Retry identik record tersimpan	200, ACK lama	200	event_id sama dengan record pertama
13	Identitas sama, nilai berbeda	409	409	Identitas pembacaan sudah dipakai untuk data berbeda
14	Kelembapan 150 % melalui MQTT	tanpa ACK	tanpa ACK	ditolak validasi subscriber

Ke-14 kasus lulus. Jumlah record tetap 2.338 sebelum dan sesudah setiap kasus, sehingga payload salah tidak masuk ke basis data dan retry tidak membuat record ganda. Selain validasi di aplikasi, CHECK dan foreign key pada tabel menjadi lapisan pengaman kedua; uji otomatis membuktikan INSERT langsung dengan suhu 200 °C atau perangkat tidak terdaftar ditolak SQLite.

4.2 Timestamp dan Kualitas Data

Dimensi	Bukti	Hasil
Completeness	Query 4: `seq_hilang` per sesi	0 nomor hilang pada 5 sesi (2.070 record bersesi)
Accuracy	Validasi rentang dan CHECK suhu/kelembapan	Semua nilai dalam rentang DHT22. Akurasi fisik tidak diklaim karena data berasal dari node demo.
Consistency	Query 4: `bukan_utc`; tipe REAL dan satuan °C / %RH	0 timestamp non-UTC; format dan satuan seragam
Timeliness	Query 4: `waktu_mundur`; Gambar 18: waktu tulis dan RTT	0 waktu mundur; median tulis lokal 4,49 ms; median RTT 9,01 ms (MQTT) dan 22,58 ms (HTTP)
Uniqueness	Query 4: `duplikat`; indeks unik identitas; uji retry	0 duplikat; retry dijawab ACK lama, konflik ditolak 409

4.3 Retensi dan Koneksi Terputus

Retensi belum dijalankan otomatis; rancangannya mengikuti materi slide 16. Data mentah disimpan 7 hari, ringkasan per jam hasil Query 2 disimpan 2 tahun, dan data lama diarsipkan ke CSV (dashboard sudah menyediakan ekspor CSV). Saat data mentah dihapus, alarm terkait ikut terhapus otomatis karena `ON DELETE CASCADE`. Untuk koneksi terputus, receiver menyimpan data ke SQLite lebih dulu sebelum sinkronisasi cloud, dan kunci idempotensi `(device_id, session_id, protokol, seq)` mencegah duplikasi saat perangkat mengirim ulang.

4.4 Uji Otomatis

Sebanyak 33 tes Python (26 tes proyek UTS dan 7 tes baru untuk basis data) serta 6 tes JavaScript dashboard lulus. Tes baru mencakup FK dan CHECK pada skema baru, pendaftaran perangkat dan alarm terparah, penolakan perangkat nonaktif melalui HTTP dan MQTT, migrasi log versi UTS tanpa kehilangan data, penanganan data lama yang melanggar aturan baru, serta halaman Database Explorer yang hanya dapat membaca.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

1. Basis data proyek UTS berhasil dikembangkan menjadi empat tabel relasional: `devices`, `readings`, `alerts`, dan `delivery_attempts`. Skemanya dilengkapi primary key, tiga foreign key, constraint CHECK dan UNIQUE, serta indeks. Skema ditulis di `database/schema.sql`, dijalankan otomatis, dan log UTS lama dimigrasikan tanpa kehilangan data.
2. Target demo Pertemuan 9 tercapai: 2.368 record tersimpan (minimal 20), timestamp konsisten dalam UTC (0 waktu mundur dan 0 non-UTC), lima query berjalan dengan hasil (minimal tiga), dan 14 payload salah ditolak tanpa menambah record.
3. Dashboard menampilkan data real-time, alarm terbaru, dan ringkasan per jam. Halaman Database Explorer menampilkan relasi, struktur tabel, serta hasil query langsung dari basis data.
4. Pada data demo lokal, rata-rata RTT MQTT (26,85 ms) lebih rendah daripada HTTP (35,83 ms). Kesimpulan umum memerlukan pengujian dengan ESP32 dan DHT22 pada jaringan sebenarnya.

5.2 Saran

- Menjalankan pengujian dengan ESP32 dan DHT22 melalui Wokwi Private Gateway atau perangkat fisik.
- Menambahkan event time absolut dari NTP pada payload ESP32.
- Menyalin tabel devices dan alerts ke Supabase, serta menjalankan job retensi terjadwal.
- Mengurangi alarm berulang dengan mencatat alarm hanya saat status berubah.

LAMPIRAN

DAFTAR BERKAS

Berkas	Isi
`database/schema.sql`	Kode CREATE TABLE, indeks, dan data awal perangkat
`database/queries.sql`	Lima query analisis yang dijalankan halaman /database
`server.py`	Ingestion MQTT/HTTP, validasi, migrasi, registrasi perangkat, dan alarm
`dashboard_state.py`	Data dashboard, struktur tabel, dan eksekusi query baca-saja
`dashboard/index.html`, `app.js`	Dashboard utama beserta panel alarm dan ringkasan per jam
`dashboard/database.html`, `database.js`	Halaman Database Explorer (relasi, struktur, query)
`tests/test_database.py`	Tes otomatis fitur basis data
`tugas/basis-data/`	Laporan ini, gambar, skrip pengambil gambar, dan bukti (database.json, validasi.json)